

Day 1

"Measuring programming progress by lines of code is like measuring aircraft building progress by weight." — Bill Gates

- Author: [Kintsugi-Programmer](#)

Disclaimer: The content presented here is a curated blend of my personal learning journey, experiences, open-source documentation, and invaluable knowledge gained from diverse sources. I do not claim sole ownership over all the material; this is a community-driven effort to learn, share, and grow together.

- [FULLSTACK_WEBDEV](#)
- [DSA_MASTERY](#)
- [CN](#)
- [DSA_LEETCODE](#)

FULLSTACK_WEBDEV

Chapter 1: Before Web Dev Journey

Chapter 1.1.: Course Roadmap Overview

- Resource: <https://www.udemy.com/course/web-dev-master/>

Main Takeaway: This course offers a structured, project-based journey from beginner to advanced, covering web fundamentals, front-end and back-end development, full-stack projects, and future topics—empowering you to build production-ready applications and stay current with evolving technologies.

1. Introduction

Welcome! This video walks through:

- How **beginners**, **intermediates**, and **advanced** learners can navigate the course
- The **current roadmap** (Phase 1: 2024) and upcoming **Phase 2** (2025) and **Phase 3** (late 2025)
- Where to find updates: downloadable attachments, website, Discord

2. Phase 1: 2024 Roadmap

2.1 Beginner Track

1. Web Development Fundamentals

- Learn without any programming prerequisites
- Understand client-server architecture, impact of AI, and trending frameworks

2. HTML Basics

- Hands-on coding challenges with instant feedback

3. CSS Mastery

- Selectors, Flexbox, Grid, production-ready projects, coding challenges

4. Utility Frameworks

- Bootstrap vs. Tailwind: differences, hands-on projects, and challenges

2.2 JavaScript Path

1. Core JavaScript

- Foundation, OOP patterns, DOM & BOM concepts

2. Advanced JavaScript & Updates

- Regularly updated advanced topics based on feedback

3. Front-End Projects

- Fun, challenging, and essential demos to solidify skills

2.3 Back-End Foundation

1. Node.js Essentials

- V8 engine, building web servers, event emitters

2. Databases

- MongoDB design, Express integration—mega full-stack project

3. Advanced Database Topics

- Aggregation pipelines, advanced queries

4. Deployment

- VPS deployment on AWS/DigitalOcean without one-click tools

2.4 Full-Stack Projects

- Q&A system with Next.js, Prisma, Neon (SQL)
- Additional production-grade applications

Navigation Tip: After completing JavaScript and back-end fundamentals, you may choose to pivot to front-end (React) before diving deeper into back-end. The recommended path is back-end first for stronger logic control.

3. Phase 2: 2025 Preview

- **Microservices Architecture:** Message queues (RabbitMQ), WebSockets, WebRTC
- **Containerization & Deployment:** Dockerizing Node.js/React, advanced VPS workflows
- **New Projects:** Two major applications recorded in early 2025

4. Phase 3: Late 2025 Preview

- **AI-Centric Development:** TensorFlow.js, machine-learning demos
- **Additional Frameworks:** Angular (latest version) based on community demand
- **Ongoing Updates:** Driven by learner feedback

5. How to Use This Roadmap

- **Beginners:** Follow the roadmap sequentially from web fundamentals to full-stack projects.
- **Intermediates:** Jump to sections of interest—Node.js, databases, or front-end frameworks.

- **Advanced Users:** Dive directly into advanced topics and full-stack projects.
 - **Stay Updated:** Check the roadmap video and pinned attachments for label changes and new releases.
-

Thank you for joining this journey. Your feedback on Discord and course reviews drives continuous improvement. Let's build the world's most comprehensive web development course—together!

Chapter 1.2.: Course Introduction: Instructor Overview and Key Highlights

Main Takeaway: Instructor Hitesh brings 15 years of software engineering and teaching experience—spanning cybersecurity, mobile and web development, leadership roles, and entrepreneurship—to deliver a single, comprehensive software engineering course that emphasizes best practices, scalability, and real-world projects.

1. Instructor Background

- **Name Pronunciation:** "Hitesh" (feel free to call him *Mister H*)
- **Teaching Experience:** 15 years of writing and teaching code
- **Student Impact:** Millions of learners coached; many have advanced from their first developer role up to senior levels ,SDE1 => SDE2 => SDE3 => Further...

2. Professional Journey

- **Cybersecurity Professional:** Launched career ensuring systems' security
- **Python Programmer → Mobile App Developer:** Transitioned into software development
- **Web Developer:** Over a decade building web applications
- **Software Engineer & Consultant:** Worked at top companies as an engineer, then adviser
- **Startup Founder & CTO:** Built and scaled a startup to acquisition; led 200-person technical team
- **Senior Director at PW (India's EdTech Unicorn):** Oversaw large-scale educational technology initiatives
- **YouTube Content Creator:**
 - Channel 1: ~1 million subscribers
 - Channel 2: ~300 thousand subscribers (growing)

3. Course Vision and Scope

- **Beyond "Web Dev":** Framed as a *software engineering* course, not just HTML/CSS/JavaScript
- **Comprehensive Curriculum:**
 - Each module acts as a standalone mini-course
 - Covers coding best practices, architecture, scalability, and real-world workflows
- **High-Quality Production:**
 - Custom subtitles for accessibility
 - Professional audio/video standards
 - Engaging narratives and storytelling

4. Teaching Philosophy

- **Simplify Complexity:** Transforms the toughest topics into easy-to-understand lessons
- **Hands-On Projects:** Real code exercises that reflect industry standards

- **Scalability Focus:**
 - All concepts and codebases built to support millions of users
 - Emphasis on performance, maintainability, and growth

5. Student Outcomes

- **Skill Mastery:** From foundational HTML/CSS to advanced software engineering principles
 - **Career Advancement:**
 - Ready for junior to senior developer roles
 - Equipped for real-world technical challenges
 - **One-Stop Resource:** Aims to be the single best global destination for aspiring web and software engineers
-

Welcome aboard this journey to master software engineering and web development under the guidance of an industry veteran who has done it all—and continues to innovate and teach at scale. Let's get started!

Chapter 1.3.: AI in Web Development: Addressing Anxiety and Embracing the Future

Key Takeaway: AI is a powerful assistant for web developers but will **not** replace the need for strong foundational skills. Embrace AI tools to accelerate routine tasks while investing in solid fundamentals to navigate complexity, collaborate effectively, and architect robust solutions.

1. Understanding AI Anxiety

Many learners worry that AI advances will render developer roles obsolete. This fear is fueled by sensational headlines, venture-capital-backed hype, and clickbait content. However, practical experience over 15+ years shows that each wave of tooling ultimately augments human capability rather than replaces it.

2. Historical Parallel: Automating Boilerplate

- **Then:** Manually writing every HTML tag and boilerplate by hand.
- **Now:** IDEs and snippet-based tools generate boilerplate instantly, freeing developers to focus on business logic.
- **Lesson:** Just as snippet tools became indispensable yet didn't replace developers, AI-powered coding assistants offer similar gains without eliminating essential human roles.

3. AI as a Developer's Assistant

AI tools excel at:

- Generating repetitive or boilerplate code
- Prototyping components and UI layouts
- Suggesting syntax corrections and refactorings
- Surface-level debugging hints

These capabilities translate to faster shipping of features and more time for high-value work.

4. Realistic Impact on Workflow

Empirical breakdown of a typical software engineer's time allocation:

- **20% Writing code**
- **30% Debugging and bugfixing**
- **50% Non-coding activities** (meetings, requirements gathering, architecture discussions)

AI primarily accelerates the 20% and partially assists with debugging, but **cannot** replace human-driven decision-making, client interaction, or strategic design.

5. The Enduring Importance of Foundations

Regardless of tooling trends—jQuery → React → tomorrow's frameworks—the core skills remain crucial:

- **HTML/CSS fundamentals**
- **JavaScript language mastery**
- **Problem-solving and algorithmic thinking**
- **Debugging methodologies**
- **Architectural design and technology evaluation**

Strong foundations enable developers to troubleshoot AI-generated code, choose appropriate technologies, and design scalable, maintainable systems.

At the end of day; "Solid Foundations" helps; rest are just tools to do work faster - Hitesh

6. Navigating the Hype

AI hype often serves marketing and funding agendas. When evaluating AI tools:

- Focus on **productivity gains** for concrete tasks.
- Verify the **accuracy** and **security** of generated code.
- Maintain **critical thinking**—always review and test AI suggestions.
- Avoid over-reliance; use AI as a complement, not a crutch.

7. Looking Ahead: AI in This Course

Later in the curriculum, you will:

- Explore specific AI-powered plugins and IDE integrations.
- Learn workflows that incorporate AI for templating, linting, and prototyping.
- Develop strategies for validating and debugging AI-assisted code.

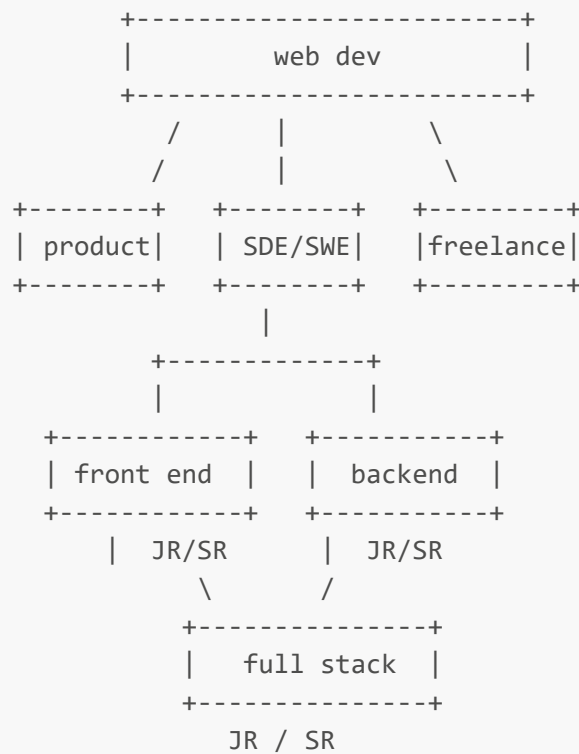
However, the immediate focus remains on building robust, tool-agnostic skills that underlie all web development.

Embrace AI to enhance your efficiency, but anchor your growth in the fundamentals—these skills will remain valuable regardless of how AI evolves.

Chaptet 1.4.: Web Development Career Pathways

Key Takeaway: Web development underpins virtually all modern software products—mobile apps like Uber, AI services like ChatGPT, SaaS platforms, and more. Mastery of front-end, back-end, databases, and DevOps

opens diverse career opportunities: freelancing, product building, salaried roles, and leadership positions, with salary potential from \$100K to \$500K+.



1. Scope of Web Development

Web Dev is just big umbrella; where all engineering branches comes - Hitesh Web development is not limited to websites—it's the backbone of nearly every digital service.

- **Mobile App Back-Ends:**
 - Account creation, OTP authentication, payment processing, and data storage for apps like Uber.
 - APIs and server logic manage data flow between client apps and databases.
- **AI Interfaces:**
 - ChatGPT's login, session management, conversation history retrieval, and UI design rely on web-development skills.
 - Advanced data handling with vector databases, embedding storage, and search functionality.

2. Core Components

1. Front End:

Design and implementation of user interfaces, interactive elements, and responsive layouts.

2. Back End:

Business logic, server-side code, API development, authentication, and integration with databases.

3. Databases:

- SQL (relational): structured schemas, complex queries
- NoSQL (document/graph): flexible schemas, high-scale reads/writes

4. DevOps & Hosting:

Deployment pipelines, containerization, cloud services (AWS, Azure, Google Cloud), monitoring, and

scaling.

3. Learning Pathways & Specializations

- **Focused Mastery:** Deep expertise in one area (e.g., front end with React/Vue, back end with Node.js/Express or Django).
- **Full Stack:** Proficiency across front end and back end to deliver end-to-end solutions and manage small teams or startups.

4. Career Routes

4.1 Freelancing

- Build custom solutions for clients (e.g., gym membership systems, salon appointment apps).
- Flexible projects, direct client interaction, variable income based on project scope and rates.

4.2 Product Development (SaaS)

- Create software products (e.g., background-removal tools, niche B2B platforms).
- Scale user base; potential for recurring revenue and high growth (example: serving 12 million users).

4.3 Salaried Employment

- **Roles & Titles:**
 - Junior Developer (SDE I): entry-level, basic project contributions
 - Senior Developer (SDE II+ / SWE): code ownership, mentorship, architecture decisions
- **Specialized Tracks:** Front End Engineer, Back End Engineer, Full Stack Engineer, DevOps Engineer.
- **Advancement:** Team Lead → Project Manager → Solutions Architect → CTO.

5. Salary Expectations

- **Range:** Approximately \$100,000 to \$200,000 median in high-purchasing-power regions; top earners up to \$500,000+.
- **Factors Influencing Salary:**
 - Geographic location and cost of living
 - High Salary Regions Mostly also have High Cost of Living
 - Company size and industry
 - Individual skill set, experience level, and niche expertise
 - Remote vs. on-site roles
- **Global Dynamics:** Remote work enables high-quality engineers in lower-cost regions to earn international market rates.

6. Realistic Outlook

IT Field is not that easy as it's looks like, lots of people left prog. just after HTML/CSS.

Real Programming starts after JS

Harder Efforts => Higher You are gonna paid, That's a Fact, Nobody can change it at all !!!

- **Entry Difficulty:** Basic HTML/CSS is foundational but not true programming; JavaScript and back-end logic mark the start of substantive development.
- **Effort vs. Reward:** Higher technical complexity and proficiency command higher compensation.
- **Continuous Learning:** Evolving technologies (e.g., vector search, cloud-native architectures) require ongoing skill development.

These notes outline the vast landscape of web development, practical real-world applications, and the multifaceted career opportunities it offers. Continuous skill enhancement and hands-on project experience are essential to unlocking higher roles and salaries.

DSA_MASTERY

Chapter 1.2.: Introduction to Data Structures Course

Main Takeaway:
This course delivers a **comprehensive, level-3 mastery** of core data structures—covering theory, analysis, and hands-on implementation in C (and convertible to C++).

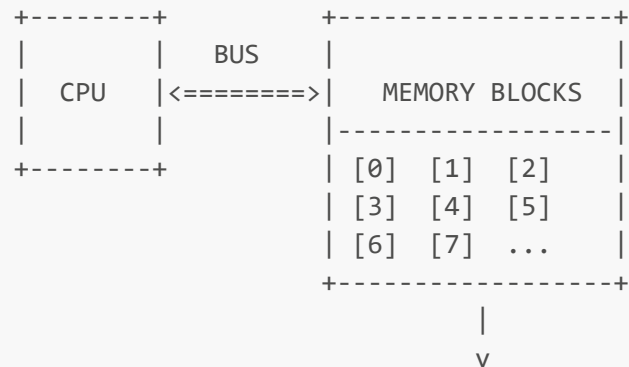
1. Course Contents

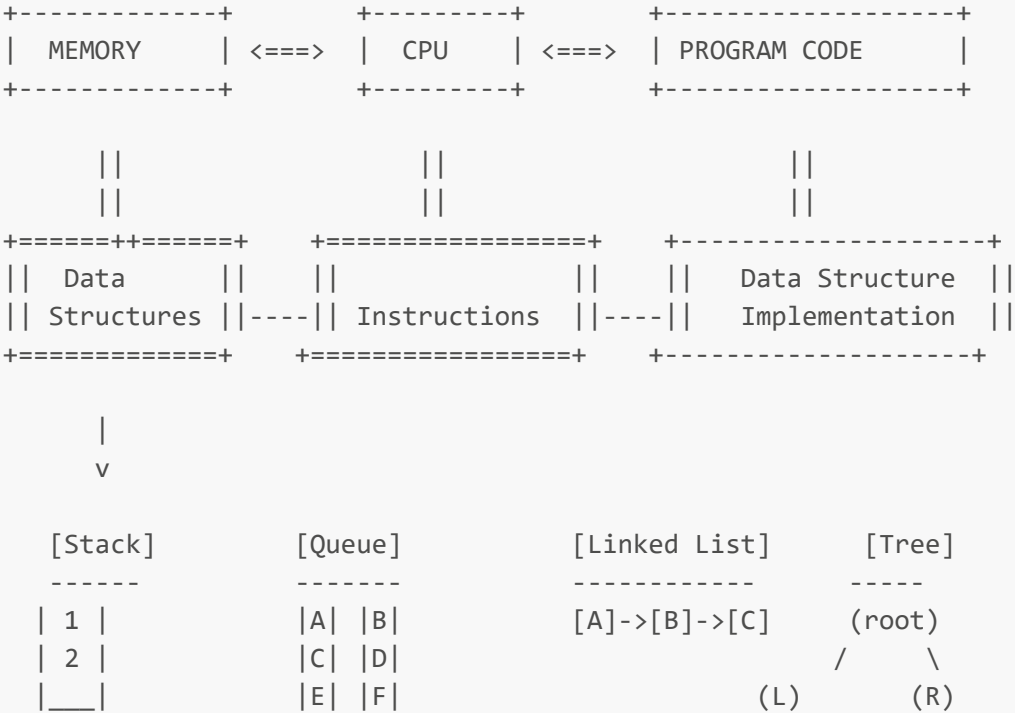
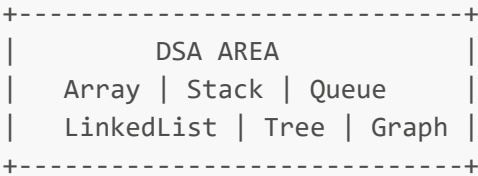
- Arrays & Matrices
- Linked List
- Stack & Queue
- Trees & Graphs
- Hashing
- Recursion
- Sorting

2. What Are Data Structures?

Program is set of instructions which performs operations on data. Without data, instructions cannot be performed. Data structures define **how** a program organizes its data in memory so that operations can be performed most **efficiently** during its execution time. They bridge the gap between:

- **Program code** (instructions)
- **Main memory** (data layout)





ETC.

Program code example:

```
-----
for (int i = 0; i < n; i++) {
    arr[i] = i * 2;
}
```

Data movement:

```
-----
[MEMORY]<--read/write-->[CPU]<--executes-->[CODE]<--organizes-->[DATA STRUCTURE]
```

3. Classification

- **Physical Data Structures** (memory layout)
 - Arrays
 - Matrices
 - Linked List
- **Logical Data Structures** (data utilization)
 - Stack, Queue, Tree, Graph, Hashing, etc.

4. Why Study Data Structures?

- **Academic requirement** for computer science and engineering students.
 - **Industry necessity:** essential for application development, performance optimization, and scalable systems.
-

5. Mastery Levels

1. **Awareness & Usage:** Know what each structure is and where to apply it.
2. **In-Depth Analysis:** Understand internal workings, operation algorithms, and perform time/space complexity analysis.
3. **Implementation Proficiency:** Code each data structure from scratch, debug, and adapt to different languages.

This course achieves **Level 3**, guiding you through theory, analysis, and complete implementations.

6. Language Choice

- **Primary:** C (no built-in data structures)
 - C is near Low-Level Lang. ,best lang. to study DSA
 - no built-in data structures thus best Lang. to understand DSA => By DIY DSA.
 - Forces clarity on every operation and memory behavior.
 - Code directly convertible to C++ (with added OO features).
 - as C is Sub-set of C++.
- **Optional Extensions:** C++, Java, C#, Python, JavaScript (all offer built-ins via STL, collections, or DOM objects).

Use C to build from first principles, then leverage language-specific collections in practice.

7. Course Organization

1. **Prerequisite Refresher**
 - C/C++ essentials: structures, functions, classes, templates, parameter passing.
 2. **Foundations of Recursion**
 - Importance in problem solving
 - Recursive vs. iterative implementations
 3. **Data Structures Modules**
 - Each topic: concept → operations → analysis → C implementation → optimization
 4. **Sorting Techniques**
 - Bubble, Selection, Insertion, Quick, Merge, Heap, etc.
 - Implementations and detailed performance analysis
-

8. Role of Recursion

Even though Recursion is usually felt as inefficient because it uses stack internally, i.e. not useful to solve large size problems; still it's imp. to study to master Problem solving skills.

Programming != Problem Solving. Programming takes weeks, its syntax. Problem solving takes Lifetime, its Maths, Maths don't have function, it has Recursion.

- Essential for mathematical problem modeling.
- Underpins many data-structure operations (e.g., tree traversals).
- Teaches problem-solving separate from language syntax.
- Course covers both recursive and loop-based implementations.
- Recursion is not used but its Supports.

9. Algorithms vs. Data Structures

- **Data Structures:** How data is stored and accessed.
- **Algorithms:** Procedures operating on data.
- This course focuses on algorithms *applied to* data structures.
- For broader algorithmic topics (graph algorithms, machine learning, etc.), refer to dedicated materials.

Next Steps

Proceed to the **Essential C/C++ Features** section to brush up on language constructs before diving into hands-on implementations.

CN

2 Computer Network Fundamentals: From Basic Communication to OSI Model

Introduction to Computer Networks

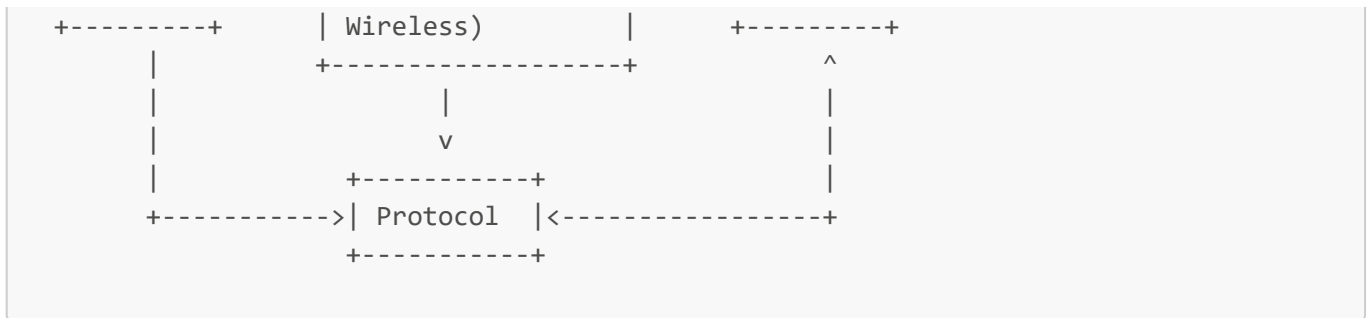
A computer network represents a collection of interconnected computing devices designed to share data and resources. The fundamental purpose of any computer network is to enable **data sharing** between various homogeneous and heterogeneous devices through established connections. This basic principle forms the foundation of all modern digital communication systems.[1][2][3][4]

Core Components of Data Communication

Essential Elements

Every computer network relies on five basic components that work together to facilitate communication:[3][4]

```
[Message]-----CONNECTION----->
+-----+           +-----+           +-----+
| Sender |----->| Transmission |----->| Receiver|
| (Device)|       | Medium (Wired/ |       | (Device)|
```



Message: The data or information that needs to be transferred from one device to another over the network. This can be text, audio, video, images, or combinations of these forms.

Sender: The device that initiates data transmission and has the information to send. This can be a computer, mobile phone, video camera, or any other computing device.[4][5]

Receiver: The destination device that expects to receive the data from the sender. Like senders, receivers can be computers, mobile phones, or other network-capable devices.[5][4]

Transmission Medium: The physical path through which data travels from sender to receiver. This includes twisted-pair cables, coaxial cables, fiber-optic cables, or wireless connections.[4]

Protocol: A defined set of rules and conventions that both sender and receiver must follow to ensure successful communication. It Gives Ability to Understand Each other. Without protocols, devices might connect physically but cannot effectively communicate.[6][4]

The Communication Process

The communication process in computer networks follows a structured approach. When a sender transmits data, it must follow specific protocols to ensure the receiver can understand and process the information correctly. This is analogous to human communication where both parties must speak the same language to understand each other effectively.[7]

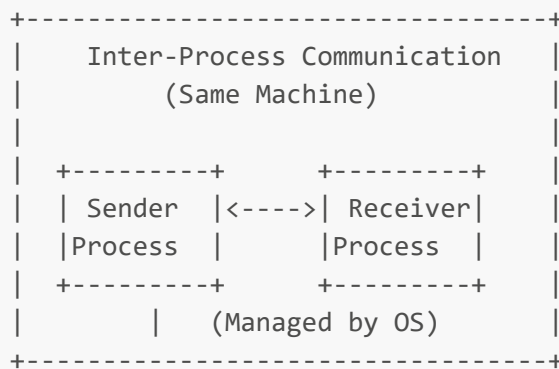
Lack of Protocol Layman Eg: Italian & Russian speaking properly, all data 100% accurate, but still can't understand each other, replace People with Machines

Inter-Process Communication vs Computer Networking

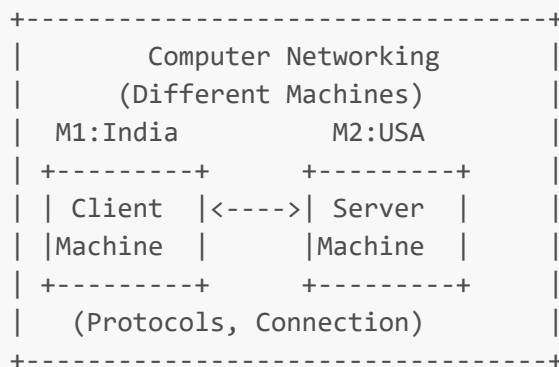
An important distinction exists between inter-process communication (IPC) and computer networking.[8][9][10][11]

Inter-Process Communication occurs when processes communicate within the same machine. For example, when you press a key on your keyboard, that input is processed and displayed on your monitor. This communication happens entirely within one system and is managed by the operating system kernel. IPC mechanisms include shared memory, message passing, pipes, and message queues, all designed for communication between processes on a single system.[10][11]

Computer Networking becomes relevant when the client and server exist on different machines that are physically separated. This separation can range from one meter to thousands of kilometers - distance is not the determining factor. The key distinction is that networking protocols are required when communication must occur between different physical machines.[9][8][6]



|| (Physical boundary) ||



Functionalities Used in Communication: Mandatory vs Optional Network Functions

Computer networks implement numerous functions, with over 70 different functionalities categorized as either mandatory or optional[12]

during Communication (Like Client Phone User to Server of Meta), these Responsibilities are **handed by Protocols** of Systems to provide relevant Functionalities for ease of communication.

all these stuff are codes, algos, loaded in our kernel, will will provide mandatory func.s

Mandatory Functions

1. Error Control: This critical function detects **whether transmitted messages arrive correctly** at their destination. Due to network noise, interference, or potential security threats, messages can be corrupted during transmission. Error control mechanisms use techniques like checksums, cyclic redundancy checks, and parity checking to **identify errors and enable retransmission when necessary**. [13][14][15]

2. Flow Control: This **manages the rate of data transmission to prevent overwhelming the receiver**. Since receiving devices have limited processing speed and memory buffers, flow control ensures that senders don't transmit data faster than receivers can handle it. This prevents buffer overflow(**Congestion**) and data loss by **putting constraints in flow**. [14][15][13]

3. Multiplexing and Demultiplexing: These transport layer functions **allow multiple applications to share a single network connection simultaneously**. Multiplexing combines data streams from

different applications into a single transmission stream using port numbers for identification.

Demultiplexing performs the reverse process, directing incoming data to the appropriate application based on port numbers.[16][17][18][19]

Optional Functions

Encryption and Decryption (Cryptography): While not required for all applications, cryptographic functions become essential for secure communications. Banking applications, secure websites (HTTPS), and other security-sensitive services require encryption to protect data from unauthorized access during transmission.[20][21][22][23]

Checkpoint Mechanisms: These functions enable resumable data transfers, particularly useful for large file downloads. When downloading a large file, checkpoints allow the process to resume from the last successful point rather than starting over if the connection fails. However, this functionality is unnecessary for small data transfers like instant messages(in Whatsapp,etc.).[24]

Importing these too will increase complexities of network, time transfer, but also enhances it at much extent, dependent on the need. Tradeoff of Security vs Speed/Simplicity.

The Need for Standardization: OSI Model

The complexity of managing over 70 different network functions necessitated the creation of standardized models. The OSI (Open Systems Interconnection) model emerged as a theoretical framework that organizes all networking functions into seven distinct layers.[25][26][27][12][6]

The Seven Layers of OSI Model

- **Physical Layer:** Handles the physical transmission of raw data bits through electrical, optical, or radio signals.[26][27]
- **Data Link Layer:** Manages node-to-node communication, error detection, and frame transmission within a single network segment.[27][26]
- **Network Layer:** Responsible for routing packets across multiple networks, logical addressing, and path determination.[28][25]
- **Transport Layer:** Provides end-to-end communication services, including error recovery and flow control. This layer implements multiplexing and demultiplexing functions.[25][16]
- **Session Layer:** Manages dialog control between applications, establishing, maintaining, and terminating connections.[26][25]
- **Presentation Layer:** Handles data formatting, encryption/decryption, and compression services.[25][26]
- **Application Layer:** Provides network services directly to end-user applications.[26][25]

Other Models also came like TCP/IP, IEEE,etc.

The Beauty of OSI Model is to organise 70+ Functionalities into just freaking 7 LAYS, WOW !!!

TCP/IP Model vs OSI Model

While the OSI model serves as a comprehensive theoretical framework, the TCP/IP model represents the practical implementation used in real-world networking.[29][30][31][32]

The TCP/IP model consists of four layers compared to OSI's seven:

- Application Layer (combines OSI's Application, Presentation, and Session layers)
- Transport Layer
- Internet Layer (equivalent to OSI's Network layer)
- Network Access Layer (combines OSI's Data Link and Physical layers)

The TCP/IP model is more reliable and widely implemented, forming the foundation of internet communications. However, the OSI model remains valuable for educational purposes and network design planning due to its detailed layer separation.[30][31][29]

Conclusion

Computer networks represent sophisticated systems designed to create seamless communication between physically separated devices. By implementing standardized protocols organized into layered models like OSI, networks can provide the illusion that remote resources are locally available. The distinction between mandatory functions (error control, flow control, multiplexing) and optional features (encryption, checkpointing) allows networks to balance functionality with performance requirements.

Understanding these fundamental concepts provides the foundation for comprehending more advanced networking topics and protocols that enable our interconnected digital world. The standardization achieved through models like OSI and TCP/IP ensures interoperability and reliability across diverse hardware and software platforms, making global communication possible.

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44

DSA_LEETCODE

1 Concatenation of Array [Easy] [Google, Adobe, Facebook]

- <https://leetcode.com/problems/concatenation-of-array/>

Ques

You are given an integer array `nums` of length `n`. Create an array `ans` of length `2n` where `ans[i] == nums[i]` and `ans[i + n] == nums[i]` for $0 \leq i < n$ (0-indexed).

Specifically, `ans` is the concatenation of two `nums` arrays.

Return the array `ans`.

Example 1:

```
Input: nums = [1,4,1,2]
```

```
Output: [1,4,1,2,1,4,1,2]
```

Example 2:

Input: nums = [22,21,20,1]

Output: [22,21,20,1,22,21,20,1]

Constraints:

1 <= nums.length <= 1000. 1 <= nums[i] <= 1000

PreReqs

```
#include<bits/stdc++.h>

int main(){
    // Vector Array
    std::vector<int> arr1; // empty vector
    std::vector<int> arr2={1,2,3}; // initialised vector
    std::vector<int> arr3(5); // size =5, each element value=default value=0
    std::cout<<arr3[0]<<std::endl; //0 // Direct Access (0based)
    std::vector<int> arr4(5,90); // size =5, each element value=90
    std::cout<<arr4[0]<<std::endl; //90
    std::cout<<arr1.size()<<std::endl; //0 // Get Length
    arr4.push_back(5); // Append // btw each push in dynamic array is O(1)
    std::cout<<arr4[5]<<std::endl; //5

    // For Loop
    for ( int i=0; i<5; i++) std::cout<<i;
    // 01234

    return 0;
}

// 0
// 90
// 0
// 5
// 0123
```

Solutions

- Basically We have to make final array of 2x input array
- We can't do just Vector Concatnation like strings
- Solution 1
 - assume nums as input vector
 - n = size of vector
 - make new arr1 of size 2n default 0
 - for loop 0 to n-1
 - arr1[i]= nums[i]

- for loop 0 to n-1
 - arr1[i+n]=arr[i]
- return arr1

```
// Solution 1
class Solution {
public:
    vector<int> getConcatenation(vector<int>& nums) {
        int n = nums.size();
        vector<int> ans(2*n);
        for (int i=0; i<n; i++){
            ans[i]= nums[i];
        }
        for (int i=0; i<n; i++){
            ans[n+i]= nums[i];
        }
        return ans;
    }
};
// Time complexity:
// O(n)
// Space complexity:
// O(n) for the output array.
```

- Solution 2
 - Iteration (One Pass)
 - assume nums as input vector
 - n = size of vector
 - make new arr1 of size 2n default 0
 - for loop 0 to n-1
 - arr1[i]= nums[i]
 - arr1[i+n]=arr[i]
 - return arr1

```
// Solution 2
class Solution {
public:
    vector<int> getConcatenation(vector<int>& nums) {
        int n = nums.size();
        vector<int> ans(2*n);
        for (int i=0; i<n; i++){
            ans[i]= nums[i];
            ans[i+n]= ans[i];
            // ans[i] = ans[i + n] = nums[i]; // we can write this too !!!
        }
        return ans;
    }
};
```

```
// Time complexity:
// O(n)
// Space complexity:
// O(n) for the output array.
```

- Solution 3 -- optimal
 - not making any new array ,saving space
 - assume nums as input vector
 - n = size of vector
 - for loop 0 to n-1
 - nums.push_back(nums[i])
 - return nums
- btw each push in dynamic array is O(1)

```
// Solution 3
class Solution {
public:
    vector<int> getConcatenation(vector<int>& nums) {
        int n = nums.size();

        for (int i=0; i<n; i++){
            nums.push_back(nums[i]);
        }

        return nums;
    }
};

// Time complexity:
// O(n)
// Space complexity:
// O(n) for the output array.
```

- Solution 4
 - Iteration (Two Pass)
 - generic sol. with times var, here =2
 - assume nums as input vector
 - n = size of vector
 - for loop 1 to times-1
 - for loop 0 to n-1
 - nums.push_back(nums[i])
 - return nums
- btw each push in dynamic array is O(1)

```
// Solution 4
class Solution {
```

```
public:
    vector<int> getConcatenation(vector<int>& nums) {
        int n = nums.size();
        int times= 2;
        for (int j=1;j<times; j++)// j is 1 because ones occurence is already here
        {
            for (int i=0; i<n; i++){
                nums.push_back(nums[i]);
            }
        }

        return nums;
    }
};

// Time complexity:
// O(n)
// Space complexity:
// O(n) for the output array.
```

2 Contains Duplicate [Easy] [Airbnb, Amazon, Apple ,Microsoft, Tcs, Google, Yahoo, Oracle, Palantir-technologies, Adobe, Uber, Facebook, Bloomberg]

- <https://leetcode.com/problems/contains-duplicate/>

Ques

Given an integer array nums, return true if any value appears more than once in the array, otherwise return false.

Example 1:

Input: nums = [1, 2, 3, 3]

Output: true

Example 2:

Input: nums = [1, 2, 3, 4]

Output: false

Constraints:

1 <= nums.length <= 105 -109 <= nums[i] <= 109

Recommended Time & Space Complexity You should aim for a solution with $O(n)$ time and $O(n)$ space, where n is the size of the input array.

Hint 1 A brute force solution would be to check every element against every other element in the array. This would be an $O(n^2)$ solution. Can you think of a better way?

Hint 2 Is there a way to check if an element is a duplicate without comparing it to every other element? Maybe there's a data structure that is useful here.

Hint 3 We can use a hash data structure like a hash set or hash map to store elements we've already seen. This will allow us to check if an element is a duplicate in constant time.

PreReqs

```
#include<bits/stdc++.h>
int main(){
    // Hashset
    std::unordered_set<int> hashSet;
    hashSet.insert(1);
    std::cout<<(
        hashSet.find(1)
        !=
        hashSet.end() // i.e if find give hashSet.end() ptr, then element doesnt
exist
    )<<std::endl; // true as 1 exists in hashset
    // 1

    // Sorting
    // Sorting in Vector
    // std::sort sorts the vector in-place and does not return a sorted vector.
    Its return type is void
    std::vector<int> v1 = {1,4,2,0};
    std::cout<<"v1: "<<v1[0]<<" "<< v1[1] << " " << v1[2] << " " << v1[3] <<
std::endl;
    // v1: 1 4 2 0
    std::sort(v1.begin(), v1.end()); // asc default
    std::cout<<"v1(sorted): "<<v1[0]<<" "<< v1[1] << " " << v1[2] << " " << v1[3]
<< std::endl;
    // v1(sorted): 0 1 2 4
    std::sort(v1.begin(), v1.end(), std::greater<int>()); // dsc
    std::cout<<"v1(sorted dsc): "<<v1[0]<<" "<< v1[1] << " " << v1[2] << " " <<
v1[3] << std::endl;
    // v1(sorted dsc): 4 2 1 0

    // unordered sets in c++ contains unique elements
    // if we even insert duplicate, it will reject it
    // we can make unord set with method to copy full the vector array, rejecting
the duplicates
```

```

v1.push_back(1);
v1.push_back(1);
std::unordered_set<int> hashSet2(v1.begin(), v1.end()) ;
std::cout<< hashSet2.size() << std::endl; // 4 // unique elements in unord set
std::cout<< v1.size() << std::endl; // 6 // duplicates elements in vect arr

    return 0;
}

// 1
// v1: 1 4 2 0
// v1(sorted): 0 1 2 4
// v1(sorted dsc): 4 2 1 0
// 4
// 6

```

Solutions

- So in this ques array
 - if any any element occurrence > 1 or have duplicates
 - return true
 - else
 - then the array have distinct elements
 - return false
- Solution 1 -- brute force
 - we are checking each element to find it's same value by traversing each array eachtime
 - given nums vect array
 - let n = nums vect array length
 - let counter = 0
 - if counter become more than 1 , i.e. item has multiple occurrence
 - for i 0->n-1
 - for j 0->n-1
 - if nums[i]==nums[j] (not i==j)
 - counter++
 - if counter>1
 - return true
 - counter = 0 // reset counter for next elements turn
 - return false // at case where counter didnt inc from 1 , i.e. not even onces occurrence

```

// Solution 1
class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        int n = nums.size();
        int counter = 0;
        for ( int i=0; i<n; i++){

```

```

        for ( int j=0; j<n; j++){
            if (nums[i]==nums[j]){
                counter++;
                if (counter> 1){
                    return true;
                }
            }
        }
        counter=0;
    }
    return false;
}
};
// // Time & Space Complexity
// // Time complexity:
// // O(n**2)
// // Space complexity:
// // O(1)

```

- Solution 2 -- optimal
 - hashset
 - efficient tc O(n)
 - directly checking if element's occurrence > 1
 - using another ds hash set
 - we are inserting in hash set one by one & parallelly checking if incoming element already exists in hash set, if it already exists, so it means at that point of time its duplicate is incoming
 - so this hashset contains duplicates
 - return true and exit, no need to continue, we got our ans
 - else try until any occurrence is duplicate
 - if not true at any case and didn't exit earlier
 - return false
 - now only 1 loop, which is even just insertion in ds is only req.
 - earlier i thought for an approach where we can remove that element in an array and still find if it exists as duplicate or not. this solution similar acts, by not deleting it but checking during genesis of array

```

// Solution 2
class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        int n = nums.size();
        unordered_set<int> hashSet;
        for ( int i=0; i<n; i++){ // or // for (int num : nums) {
            if (hashSet.find(nums[i])!=hashSet.end()){ // or // if
(find.count(num)) {
                return true;
            }
            else {
                hashSet.insert(nums[i]);
            }
        }
    }
}

```

```

    }
    }
    return false;
}
};
// Time & Space Complexity
// Time complexity:
// O(n)
// Space complexity:
// O(n)

```

- Solution 3
 - 2 Pointer Approach & Sorting
 - no need of new space
 - sort the array
 - $O(n \log n)$
 - then the duplicates would be adjacent to each other
 - even even once 2 adjacent elements are same, then we got our duplicate array proof
 - do one loop to just check if `arr[i] == arr[i+1]`
 - if true, return true
 - if loops ends without returning true; then array is distinct
 - return false

```

// Solution 3
class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        int n = nums.size(); // or just use the method in loop too
        sort(nums.begin(), nums.end());
        for (int i=0; i<n-1; i++){ // we took n-1 as to not going to case (where
i=n-1 index(last index), i+1=n index(exceeding limit)), and now at i= n-2 (2nd
last element index), then i+1 = n-1(last element index) :)
            if( nums[i]==nums[i+1]) return true;
        }
        return false;
    }
};
// Time & Space Complexity
// Time complexity:
// O(n log n)
// Space complexity:
// O(1) or O(n) depending on the sorting algorithm.

```

- Solution 4 -- optimal
 - Hash Set Length
 - we can just make set of distinct elements and compare size of old array, if same, then false, else true

- unordered sets in c++ contains unique elements
 - if we even insert duplicate, it will reject it
- we can make unord set with method to copy full the vector array, rejecting the duplicates
- if set size < vector size, it means that vec had duplicate elements
 - return true
- else return false

```
// Solution 4
class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        return (
            unordered_set<int>(nums.begin(), nums.end())
                .size()
                <
                nums.size()
        );
    }
};
// Time & Space Complexity
// Time complexity:
// O(n)
// Space complexity:
// O(n)
```

End-of-File

The [KintsugiStack](#) repository, authored by Kintsugi-Programmer, is less a comprehensive resource and more an Artifact of Continuous Research and Deep Inquiry into Computer Science and Software Engineering. It serves as a transparent ledger of the author's relentless pursuit of mastery, from the foundational algorithms to modern full-stack implementation.

Made with ❤️ [Kintsugi-Programmer](#)